

Guía completa para desarrollar un *Data Field* de Connect IQ orientado a nutrición (aplicación INNER)

Esta guía resume la información oficial de Garmin y experiencias de desarrolladores para crear un **Data Field** de Connect IQ que calcule, muestre y registre avisos de nutrición durante actividades de ciclismo. El objetivo es trasladar la lógica de la aplicación INNER (descarga del plan de alimentación y avisos en tiempo real) a un *data field* que se integre con las pantallas nativas de Garmin sin perder funcionalidades.

1 Conceptos básicos de los Data Fields

- **Qué son:** un data field es un complemento que se integra en las pantallas de actividad de Garmin y calcula nuevos valores a partir de los datos de sensores. No es una aplicación independiente; se añade a un perfil de actividad existente y comparte el bucle de pantallas de Garmin ¹.
- **Frecuencia de cálculo:** al implementar la clase `WatchUi.DataField` se debe sobrescribir el método `compute(info)`. Este método recibe un objeto `Activity.Info` con métricas como velocidad, cadencia, potencia, FC, altitud, calorías, etc., y se invoca una vez por segundo ². **No inicializar** variables dentro de `compute()` porque se ejecuta de forma asíncrona junto a `onUpdate()` ³.
- **Tipos de Data Field:**
 - **SimpleDataField:** para mostrar un único valor junto con una etiqueta; Garmin gestiona el diseño por defecto ⁴.
 - **DataField complejo:** permite dibujar múltiples métricas y personalizar el diseño usando `dc.draw` o archivos de recursos (`layouts.xml`). Debe adaptarse a los distintos layouts de 1-4 campos y a diferentes resoluciones.
 - **Distribución en pantalla:** los data fields se colocan dentro de un cuadro rectangular asignado por el usuario según el layout (1 campo, 2 campos, etc.). Deben dibujar únicamente dentro de ese espacio; no pueden crear nuevas páginas ni ocupar toda la pantalla ⁵.
 - **Obscuridad de bordes:** en pantallas redondas o con bisel prominente, algunas esquinas del rectángulo pueden quedar ocultas. El método `getObscurityFlags()` indica qué bordes están tapados y solo es válido durante `onUpdate()` ⁶. La interfaz debe ajustarse en consecuencia ⁷.

2 Diseño y presentación

2.1 Diseño de la pantalla

- **Layout adaptable:** el data field debe escalar su contenido para los layouts de 1, 2, 3 y 4 campos. Para un diseño uniforme, Garmin proporciona una **tabla de layouts** en el SDK (`devices.xml`) con coordenadas, tamaños de fuente y colores; sin embargo, muchos desarrolladores prefieren dibujar manualmente (`dc.drawText`, `dc.drawBitmap`) y ajustar dinámicamente en

`onUpdate()` . Algunos foros indican que replicar el estilo nativo requiere ajustar fuentes y coordenadas según el dispositivo ⁸ .

- **Uso de recursos:** alternar entre recursos gráficos (`layouts.xml`) y dibujo manual. Los recursos permiten mantener un diseño consistente y reutilizable ⁹ , pero consumen algo más de memoria. Dibujar manualmente ofrece más control y optimización.
- **Colores y fuentes:** para un aspecto profesional, seguir las gamas cromáticas de Garmin y de la marca INNER. Las **Directrices de marca de Connect IQ** recomiendan íconos simples, con fondo sólido y sin texto ¹⁰ .
- **Campos de datos y etiquetas:** en un data field complejo puede haber varias métricas: tiempo hasta la siguiente ingesta, gramos de carbohidratos consumidos, hidratación restante, etc. Utilice diferentes tamaños de fuente o colores para destacar información importante (p. ej., rojo cuando se ha retrasado una ingesta).

2.2 Alertas y notificaciones

- **Alertas en pantalla completa:** la guía de usuario especifica que, cuando el campo de datos no está visible, puede lanzar una **alerta** a pantalla completa (`showAlert`) a través de un `DataFieldAlert` . El usuario debe habilitar manualmente las alertas de campos Connect IQ en el menú de la actividad (Activity > Alarms > Connect IQ) ¹¹ . Las alertas se pueden descartar manualmente o se cierran tras un periodo.
- **Toasts:** desde API 3.4.0 existe `WatchUi.showToast()` , que muestra una notificación breve con un icono opcional ¹² . Es menos intrusivo que un `showAlert` y útil para avisos breves como “toma 20 g de carbohidratos”.
- **Vibración y tonos:** para llamar la atención se puede usar el módulo `Attention` . La API `Attention.vibrate()` activa el motor de vibración, `Attention.playTone()` reproduce un tono y `Attention.backlight()` o `Attention.setFlashlightMode()` activan la luz según el dispositivo ¹³ . Debe usarse con moderación para evitar incomodar al usuario o agotar la batería.

3 Configuración y almacenamiento de datos

3.1 Ajustes de usuario

- **Sin entrada directa:** los data fields **no permiten introducir datos directamente**. La configuración se realiza mediante la app **Garmin Connect** o Garmin Express. La guía señala que los campos pueden proporcionar un flujo de configuración en el dispositivo en dispositivos System 4 o superior, accesible desde el menú de la actividad ¹⁴ .
- **Framework de ajustes:** se define un par **Property / Setting** en el fichero de recursos (`strings.xml`), y se asocia mediante la etiqueta `<setting>` ; el valor se almacena en la property y puede ser consultado con `AppBase.getProperty()` o modificado con `setProperty()` ¹⁵ . Los tipos de ajustes admiten listas, booleanos, numéricos y arrays ¹⁶ .
- **Interfaz en el dispositivo:** `AppBase.getSettingsView()` permite crear una interfaz de ajustes en el dispositivo para watch faces y data fields (API ≥ 3.2). Devuelve un `View` y un delegado de entrada. No todos los dispositivos soportan esta interfaz ¹⁷ y hay informes de que en algunos modelos (por ejemplo, Venu 2) no aparece la opción de “Connect IQ Fields” ¹⁸ .

3.2 Persistencia de datos

- **Storage:** `Application.Storage` permite guardar datos de uso frecuente (p. ej., el plan de nutrición descargado) y recuperarlos en sesiones posteriores. Las claves y valores están limitados a 8 kB cada una y el total de almacenamiento a 128 kB ¹⁹ . Se puede acceder a este

almacenamiento desde procesos de fondo; el callback `AppBase.onStorageChanged()` avisa cuando se modifican datos ²⁰.

- **Properties y Settings:** los valores de las propiedades (definidos en recursos) se pueden establecer vía Garmin Connect. Son adecuados para pequeñas configuraciones como seleccionar la fórmula de cálculo o el número de ingestas.

4 Comunicación con servicios externos

4.1 Límites generales

- **Comunicación en data fields:** la versión API 5.0.0 (System 7) habilitó que algunos data fields puedan usar APIs de `Communications` directamente, **pero solo cuando el campo está en primer plano** ²¹. En versiones anteriores, el data field no tiene acceso directo a `makeWebRequest()`, y la comunicación debe realizarse mediante un **servicio en segundo plano** ²².
- **Servicios de fondo:** al anotar un archivo con `:background` se crea un servicio independiente que puede ejecutarse aunque el campo de datos no esté visible. Se pueden registrar eventos temporales (temporal events) que se disparan cada 5 minutos como mínimo y el servicio puede ejecutarse **máximo 30 segundos** ²² ²³. Dentro de ese servicio se puede llamar a `Communications.makeWebRequest()` o `Communications.makeImageRequest()` para descargar datos y almacenar el resultado usando `Background.exit()` y el callback `onBackgroundData()` ²⁴.

4.2 Descarga del plan de nutrición y comunicación con la web de INNER

1. **Sincronización previa:** la opción más robusta es transferir el plan de nutrición a través de los ajustes de la app en Garmin Connect. El usuario configura su plan en la web de INNER y la app móvil actualiza el fichero de configuración; el data field lee el JSON almacenado en `property` o `Storage`. Esto garantiza compatibilidad con todos los dispositivos y evita restricciones de temporal events.
2. **Actualización durante la actividad:**
 3. Para API < 5.0.0: implemente un `ServiceDelegate` marcado con `:background`. Registre un evento temporal con `Background.registerForTemporalEvent()` al inicio de la actividad. Cada 5 minutos (o el intervalo deseado) el servicio se despierta, ejecuta `makeWebRequest()` para comprobar si hay cambios en el plan (o nuevas recomendaciones) y almacena los datos en `Storage`. El servicio debe llamar a `Background.exit()` antes de 30 segundos para no ser finalizado ²³.
 4. Para API ≥ 5.0.0 y cuando el usuario está viendo la pantalla: se puede ejecutar `makeWebRequest()` directamente desde el data field, pero hay que tener en cuenta que no funcionará cuando el campo no sea visible. Por lo tanto, aún conviene mantener un servicio de fondo para fiabilidad.
5. **Gestión del estado y notificaciones:** el resultado de la descarga se almacena en `Storage`, y el data field actualiza su estructura interna en `onStorageChanged()`. Tras procesar los nuevos datos, se pueden lanzar alertas con `showAlert()` o toasts para avisar al usuario de la próxima ingesta.

5 Cálculo de ingestión y avisos

5.1 Uso de Activity.Info

El objeto `Activity.Info` que se recibe en `compute()` proporciona métricas como: - Ritmo/velocidad, distancia, tiempo, - Potencia, cadencia, - Calorías totales y en el paso, - Altitud y pendiente, - Pulsaciones y zona de frecuencia cardíaca ²⁵.

Estas métricas se utilizan para estimar el consumo de energía y el gasto de carbohidratos. Por ejemplo:

- **Consumo calórico:** multiplicar la potencia por el tiempo y dividir por una constante de conversión ($1\text{ W} \approx 0,239\text{ kcal/s}$).
- **Glucógeno disponible:** inicializar con el plan predescargado y restar el gasto acumulado.
- **Hidratación:** estimar la sudoración en función de la potencia, temperatura y duración; o basarse en un valor fijo por hora.

`compute()` debería actualizar variables de estado (tiempo desde la última ingesta, gramos restantes, etc.) y devolver el valor principal a mostrar (p. ej., tiempo hasta próxima ingesta). En `onUpdate(dc)` se dibujan los valores auxiliares.

5.2 Registro en el archivo FIT

Para que los datos de nutrición estén disponibles en Garmin Connect después del entrenamiento, utilice el módulo `FitContributor`:

- Llamar a `createField(name, fieldId, type, options)` en el constructor del data field para definir campos FIT personalizados (por ejemplo, `CarbIntake`, `FluidIntake`, `GelsConsumed`). Es necesario proporcionar un ID único, tipo de datos (enteros, float o string) y unidades opcionales ²⁶.
- Cada vez que se ingiera un alimento o bebida, actualice el valor de ese campo antes de regresar en `compute()`; se registrará en el FIT a razón de 1 Hz.
- Recuerde que existe un límite de **16 campos FIT en total** por actividad (sumando todos los data fields). Añadir demasiados campos puede provocar fallos de memoria; se recomienda permitir al usuario activar o desactivar los campos que no necesite ²⁷.

5.3 Alertas y cálculo del plan

1. **Plan secuencial:** el plan predescargado contendrá los tiempos y cantidades de ingesta (por ejemplo, "00:45 h – 30 g carbohidratos", "01:15 h – 500 ml agua"). El data field mantiene un índice del próximo evento. En cada llamada a `compute()` se comprueba el tiempo transcurrido; cuando se aproxima un punto de ingesta, se lanza un `showAlert()` con el mensaje correspondiente. Si el usuario no abre la pantalla, el aviso sigue apareciendo sobre la pantalla actual (si las alertas están activadas).
2. **Plan dinámico:** para personalizar en tiempo real, se pueden ajustar los tiempos en función del consumo de energía (por ejemplo, adelantar la ingesta si la potencia media es más alta). Esto se calcula comparando el gasto calórico real con el plan previsto.
3. **Registro manual de la ingesta:** como los data fields no aceptan botones específicos, se puede usar el botón `lap` (`KEY_LAP`) o un gesto configurable para que el usuario indique que ha consumido un gel o bebida. Este evento actualizaría el estado interno y el FIT. Otra opción es

incluir un *long-press* en la propia pantalla ($API \geq 5.0.0$) o utilizar la interfaz en el teléfono para registrar consumos.

6 Optimización de memoria y rendimiento

- **Memoria disponible:** los data fields tienen límites estrictos; dispositivos antiguos como Fēnix 5 disponen de ~28 kB, mientras que los Edge 1030 llegan a 128 kB y los relojes modernos hasta 256 kB ²⁷. El código (archivo .prg) no puede exceder la memoria disponible.
- **Buenas prácticas:** foros de desarrolladores recomiendan reducir el uso de recursos y optimizar el código: evitar objetos temporales, usar arrays en lugar de diccionarios, reemplazar enums por constantes, minimizar cadenas de texto, desactivar `System.println`, utilizar fuentes nativas y eliminar bitmaps innecesarios ²⁸. Para reducir el tamaño del FIT, limite el número de campos personalizados y no registre datos que no aporten valor.
- **Dibujo eficiente:** en `onUpdate(dc)` sólo redibujar cuando cambien los valores. Cachear posiciones y formatos en variables inicializadas en `initialize()` para no recalcular en cada actualización.

7 Manifiesto y permisos

- **Manifest.xml:** el archivo de manifiesto define el tipo (`datafield`), la versión mínima de API (`minApiLevel`), las bibliotecas utilizadas y los dispositivos soportados (bloque `<iq:product>`). Para restringir el data field únicamente a actividades de ciclismo, utilice el filtro `<iq:activityFilter>` con `sport="Toybox.Activity.SPORT_CYCLING"` ²⁹. Esto evitará que aparezca en actividades no relacionadas.
- **Permisos:** añadir `<iq:permissions>` con `Communications` para realizar peticiones web y `Background` para usar servicios en segundo plano. Incluir también `FitContributor` si se van a grabar campos en el archivo FIT.
- **Revisión y política:** Garmin revisa las aplicaciones antes de publicarlas. Las directrices exigen que la app no drene la batería ni sobreescriba datos del usuario y que cumpla con la normativa de privacidad y propiedad intelectual ³⁰. También se debe cumplir el RGPD en el tratamiento de datos personales.

8 Distribución y pruebas

1. **Simulación:** utilice el emulador de Connect IQ para probar el data field en distintos dispositivos y layouts. Verifique que la información se dibuja correctamente, que las alertas se muestran y que la memoria utilizada no excede los límites.
2. **Exportación:** en el entorno de desarrollo (Eclipse con el plugin de Garmin o Visual Studio Code con herramientas CIQ) ejecute **Monkey C → Export Project**. Este asistente genera un archivo `.iq` con los binarios específicos para cada dispositivo ³¹.
3. **Publicación:** suba el archivo `.iq` a la Connect IQ Store a través del formulario "Submit an App". Asegúrese de adjuntar:
 4. Icono de la aplicación (500×500 px) siguiendo las guías de marca ¹⁰.
 5. Capturas de pantalla y descripción clara del funcionamiento (indicar que hay que activar las alertas de Connect IQ y cómo añadir el campo a una pantalla de la actividad).
 6. Política de privacidad y detalles sobre el tratamiento de datos.

7. **Pruebas beta:** Garmin ofrece un programa beta que permite distribuir versiones preliminares a usuarios para obtener feedback antes de la publicación oficial. Es recomendable utilizarlo para validar el cálculo de ingestiones y la estabilidad del servicio de fondo.

9 Flujo recomendado para INNER

1. **Migrar la lógica a un Data Field complejo:** utilice un diseño que muestre los datos esenciales de nutrición (tiempo restante para la próxima ingesta, gramos de carbohidratos acumulados, hidratos pendientes) y un pequeño gráfico o barra que represente el porcentaje de cumplimiento. El usuario verá esta pantalla en el layout de una o dos casillas, según prefiera.
2. **Sincronizar el plan:** al crear o modificar el plan en la web de INNER, la app móvil actualiza la configuración del data field en Garmin Connect. Al iniciar el entrenamiento, el data field lee el plan completo desde `Storage` o las propiedades. Esto evita depender de la conectividad durante la actividad y asegura que el plan esté disponible incluso en dispositivos sin API 5.0.0.
3. **Implementar un servicio de fondo** (opcional): si se desea actualizar el plan o enviar mensajes dinámicos durante la actividad, se configura un `ServiceDelegate` que despierte cada 5 minutos (límite de Garmin) y llame a la API web de INNER usando `makeWebRequest()`. El servicio guarda los cambios y finaliza antes de 30 segundos²². El data field recibe los datos mediante `onBackgroundData()` y actualiza el estado.
4. **Calcular consumo y programar alertas:** en `compute(info)` utilice la potencia, la frecuencia cardíaca y las calorías para estimar el gasto energético. Mantenga una estructura de plan (lista de ingestas) y un índice del próximo aviso. Cuando el tiempo restante sea menor a un umbral (p. ej., 2 min), llame a `showAlert()` con un `DataFieldAlert` que muestre la cantidad a ingerir y active `Attention.vibrate()` para asegurar que el usuario lo perciba. Entre ingestas, muestre en pantalla el tiempo restante y el balance energético actual.
5. **Registrar ingestas:** al detectar un evento de ingesta (manual o programado), actualice el estado interno y el campo FIT correspondiente. El plan se ajusta si el usuario adelanta o retrasa una ingesta. Al final del entrenamiento, el usuario podrá analizar en Garmin Connect los datos registrados.
6. **Optimizar y probar:** revise la memoria ocupada, pruebe en múltiples dispositivos y layouts, verifique la precisión del cálculo en distintas intensidades y asegure que las alertas no saturan al usuario. Ajuste la frecuencia de vibración y el tono para que sean breves y eficaces.

10 Conclusiones

Desarrollar un data field profesional para la aplicación INNER implica comprender las limitaciones y capacidades del ecosistema Connect IQ. La clave es **aprovechar las APIs de Garmin** para obtener métricas en tiempo real, **gestionar la sincronización de datos** a través de los ajustes y servicios en segundo plano, y **ofrecer una experiencia de usuario fluida** mediante alertas bien diseñadas y un diseño de pantalla adaptable. Siguiendo las directrices oficiales de Garmin y adoptando buenas prácticas de optimización y testeo, se puede crear un data field robusto que ayude a los ciclistas a gestionar su nutrición sin renunciar a las pantallas nativas de su dispositivo.

1 **Connect IQ Basics**

<https://developer.garmin.com/connect-iq/connect-iq-basics/app-types/>

2 6 26 **Toybox.WatchUi.DataField**

<https://developer.garmin.com/connect-iq/api-docs/Toybox/WatchUi/DataField.html>

3 4 **Toybox.WatchUi.SimpleDataField**

<https://developer.garmin.com/connect-iq/api-docs/Toybox/WatchUi/SimpleDataField.html>

5 **Field layouts intro: What are they and how to use them? - Connect IQ App Development Discussion - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/f/discussion/409338/field-layouts-intro-what-are-they-and-how-to-use-them>

7 11 14 **User Experience Guidelines**

<https://developer.garmin.com/connect-iq/user-experience-guidelines/data-fields/>

8 **Simple DataField layout specs - Connect IQ App Development Discussion - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/f/discussion/7650/simple-datafield-layout-specs>

9 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/layouts/>

10 **Connect IQ | Garmin Brand Guidelines | Garmin Developers**

<https://developer.garmin.com/brand-guidelines/connect-iq/>

12 **Toybox.WatchUi**

<https://developer.garmin.com/connect-iq/api-docs/Toybox/WatchUi.html>

13 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/getting-the-users-attention/>

15 16 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/properties-and-app-settings/>

17 **Toybox.Application.AppBase**

<https://developer.garmin.com/connect-iq/api-docs/Toybox/Application/AppBase.html>

18 **AppBase.getSettingsView does not work for Venu 2 - Connect IQ Bug Reports - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/i/bug-reports/appbase-getsettingsview-does-not-work-for-venu-2>

19 20 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/persisting-data/>

21 **Welcome to System 7 - News & Announcements - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/b/news-announcements/posts/welcome-to-system-7>

22 24 **Connect IQ FAQ**

<https://developer.garmin.com/connect-iq/connect-iq-faq/how-do-i-create-a-connect-iq-background-service/>

23 **Background Network Communication Capability in Connect IQ - Connect IQ App Development Discussion - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/f/discussion/416611/background-network-communication-capability-in-connect-iq>

25 **Toybox.Activity.Info**

<https://developer.garmin.com/connect-iq/api-docs/Toybox/Activity/Info.html>

27 **Device Memory Limits - Connect IQ App Development Discussion - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/f/discussion/418612/device-memory-limits>

28 **Best practices for reducing memory usage? - Connect IQ App Development Discussion - Connect IQ - Garmin Forums**

<https://forums.garmin.com/developer/connect-iq/f/discussion/252057/best-practices-for-reducing-memory-usage>

29 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/manifest-and-permissions/>

30 **App Review Guidelines**

<https://developer.garmin.com/connect-iq/app-review-guidelines/>

31 **Core Topics**

<https://developer.garmin.com/connect-iq/core-topics/publishing-to-the-store/>